

CS4001 — Introduction to Object-Oriented Programming

Coursework Assignment

GadgetShop — Java Swing Inventory Manager

Student Name: MD SAZZAD HOSSAIN SAYON

Student ID: 221LM2590017

Module: CS4001 — Introduction to OOP

GitHub Repository:

<https://github.com/sazzadLM2590017/CS4001-Introduction-to-Object-Oriented-Programming.git>

1. Introduction

This report presents the design, implementation, and testing of a Java Swing desktop application named GadgetShop, developed as part of the CS4001 coursework. The application simulates an inventory management system for a gadget shop, enabling users to add, view, and interact with two product categories: mobile phones and MP3 players.

The system is implemented using four Java classes — Gadget, Mobile, MP3, and GadgetShop — and demonstrates key object-oriented programming concepts such as inheritance, encapsulation, and polymorphism. The graphical user interface is built using Java Swing components and handles user interactions through an ActionListener.

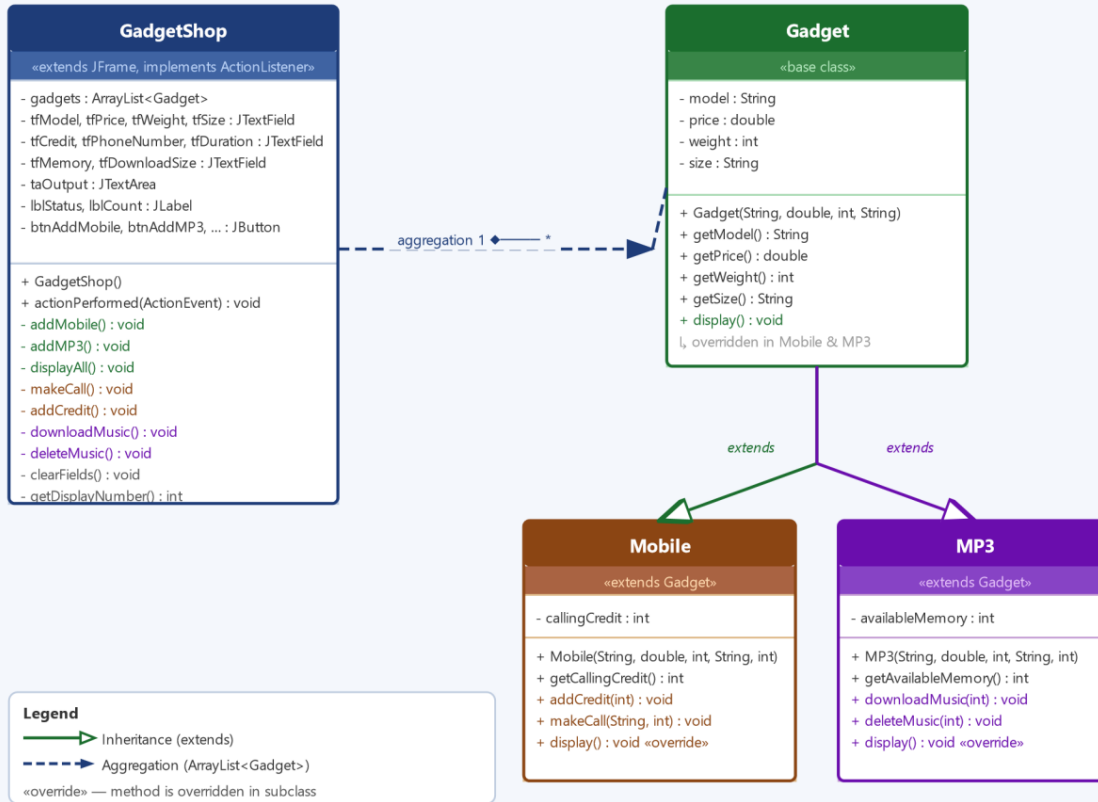
All gadget objects are stored in an `ArrayList<Gadget>`, allowing both Mobile and MP3 instances to be managed within a single collection using runtime polymorphism.

2. Class Diagram

The UML class diagram below illustrates the four classes and their relationships. Mobile and MP3 both extend the base class Gadget. GadgetShop extends JFrame and implements ActionListener, and maintains an aggregation relationship with Gadget via an ArrayList.

UML Class Diagram — GadgetShop CS4001

MD GALIB SHIFULLAH | Student ID: 221LM25913



Class hierarchy summary:

- Gadget — base class (model, price, weight, size)
- Mobile extends Gadget — adds callingCredit, makeCall(), addCredit()
- MP3 extends Gadget — adds availableMemory, downloadMusic(), deleteMusic()
- GadgetShop extends JFrame, implements ActionListener — stores ArrayList<Gadget>

3. Method Descriptions

3.1 Gadget.java

Method Signature	Description
Gadget(String, double, int, String)	Constructor. Assigns model, price, weight, and size to their respective private fields.
getModel() : String	Returns the model name of the gadget.
getPrice() : double	Returns the price of the gadget in pounds.
getWeight() : int	Returns the weight of the gadget in grams.
getSize() : String	Returns the size/dimensions string (e.g. "71mm x 137mm x 9mm").
display() : void	Prints model, formatted price, weight, and size to System.out. Overridden by subclasses.

3.2 Mobile.java

Method Signature	Description
Mobile(String, double, int, String, int)	Constructor. Calls super() with first four params, assigns callingCredit.
getCallingCredit() : int	Returns the remaining calling credit in minutes.
addCredit(int amount) : void	Adds amount to callingCredit if amount > 0; otherwise prints an error message.

makeCall(String phoneNumber, int duration) : void	Deducts duration from callingCredit if sufficient; otherwise prints an insufficient credit error.
display() : void	Calls super.display() then prints remaining callingCredit. Overrides Gadget.display().

3.3 MP3.java

Method Signature	Description
MP3(String, double, int, String, int)	Constructor. Calls super() with first four params, assigns availableMemory.
getAvailableMemory() : int	Returns the remaining available memory in MB.
downloadMusic(int memoryRequired) : void	Deducts memoryRequired from availableMemory if sufficient; otherwise prints an error.
deleteMusic(int memoryFreed) : void	Adds memoryFreed back to availableMemory. Always succeeds.
display() : void	Calls super.display() then prints remaining availableMemory. Overrides Gadget.display().

4. Pseudocode

The following pseudocode describes the six key algorithms implemented in this application.

4.1 Getting the Display Number from the GUI

```

FUNCTION getDisplayNumber() RETURNS integer
    SET displayNumber = -1
    TRY
        SET input = INTEGER( tfDisplayNumber.getText() )
        IF input >= 0 AND input < gadgets.size() THEN
            SET displayNumber = input
        ELSE
            SHOW WARNING dialog: "Display number out of range."
        END IF
    CATCH NumberFormatException
        SHOW ERROR dialog: "Display number must be a whole number."
    END TRY
    RETURN displayNumber
END FUNCTION

```

4.2 Adding a Mobile Phone

```

PROCEDURE addMobile()
    SET m = NEW Mobile(readModel(), readPrice(), readWeight(), readSize(), readCredit())    CALL gadgets.add(m)
    PRINT "Mobile added at index " + (gadgets.size() - 1)
    CALL captureDisplay(m)
END PROCEDURE

```

4.3 Adding an MP3 Player

```

PROCEDURE addMP3()
    SET mp3 = NEW MP3(readModel(), readPrice(), readWeight(), readSize(), readMemory())
    CALL gadgets.add(mp3)
    PRINT "MP3 added at index " + (gadgets.size() - 1)
    CALL captureDisplay(mp3)
END PROCEDURE

```

4.4 Displaying All Gadgets

```

PROCEDURE displayAll()
    IF gadgets is empty THEN
        PRINT "No gadgets in the shop yet."
        RETURN
    END IF
    PRINT "=== All Gadgets ==="
    FOR i = 0 TO gadgets.size() - 1 DO        PRINT "Gadget #" + i
        REDIRECT System.out TO ByteArrayOutputStream
    END FOR

```

```

CALL gadgets.get(i).display()
RESTORE System.out
APPEND captured output TO taOutput
PRINT separator line
END FOR
END PROCEDURE

```

4.5 Making a Call

```

PROCEDURE makeCall()
    SET idx = getDisplayNumber()
    IF idx == -1 THEN RETURN END IF
    SET g = gadgets.get(idx)
    IF g is NOT instance of Mobile THEN
        PRINT "Error: not a Mobile phone."
        RETURN
    END IF
    SET ph = readPhoneNumber()
    SET dur = readDuration()
    REDIRECT System.out TO ByteArrayOutputStream
    CALL ((Mobile) g).makeCall(ph, dur)
    RESTORE System.out
    APPEND captured output TO taOutput END
PROCEDURE

```

4.6 Downloading Music

```

PROCEDURE downloadMusic()
    SET idx = getDisplayNumber()
    IF idx == -1 THEN RETURN END IF
    SET g = gadgets.get(idx)
    IF g is NOT instance of MP3 THEN
        PRINT "Error: not an MP3 player."
        RETURN
    END IF
    SET ds = readDownloadSize()
    REDIRECT System.out TO ByteArrayOutputStream
    CALL ((MP3) g).downloadMusic(ds)
    RESTORE System.out
    APPEND captured output TO taOutput END
PROCEDURE

```

5. Class Relationships

The application uses an inheritance hierarchy. Both Mobile and MP3 are subclasses of Gadget, inheriting its four attributes (model, price, weight, size) and the display() method without code duplication.

GadgetShop extends JFrame (from Java Swing) and implements ActionListener to handle button click events. It holds an ArrayList<Gadget> which stores both Mobile and MP3 objects through polymorphism — both types are treated uniformly as Gadget references until a specific subclass operation is required.

6. Encapsulation

All attributes in every class are declared private. External code cannot read or modify them directly; it must use the public accessor methods (getModel(), getPrice(), etc.) provided by each class. This prevents accidental corruption of data and ensures that any future validation logic can be added inside the class without changing external code.

For example, callingCredit in Mobile can only be reduced by calling makeCall(), which enforces the rule that credit cannot go negative. Similarly, availableMemory in MP3 can only be reduced by downloadMusic(), which first checks that sufficient memory exists.

7. GUI Design

The graphical interface is built entirely with Java Swing and is divided into three main areas:

1. Gadget Details Panel — a scrollable left panel containing the 10 text fields, grouped into colourcoded sections: Gadget Details (blue), Mobile Phone (amber), MP3 Player (purple), and Action Target (green).
2. Output Panel — a read-only, scrollable JTextArea on the right that captures and displays the results of all operations.
3. Button Panel — three rows of buttons at the bottom: Add/View row (Add Mobile, Add MP3, Display All), Actions row (Make A Call, Add Credit, Download Music, Delete Music), and a full-width red Clear button.

A key design challenge was that `display()` writes to `System.out`. To show this output in the GUI, `System.out` is temporarily redirected to a `ByteArrayOutputStream` before each call, the text is captured, and `System.out` is restored. The window supports Full Display (maximised) and Compact (780×580) view modes via buttons in the header.

8. Polymorphism

Polymorphism is applied in two ways:

Method overriding: Both Mobile and MP3 override the `display()` method inherited from Gadget. When `display()` is called on an object stored as a Gadget reference inside `displayAll()`, Java automatically dispatches to the correct overridden version at runtime.

ArrayList of base type: The `ArrayList<Gadget>` stores both Mobile and MP3 objects. A single `gadgets.get(i).display()` call in `displayAll()` correctly dispatches to the right subclass method without any type checking — demonstrating runtime polymorphism.

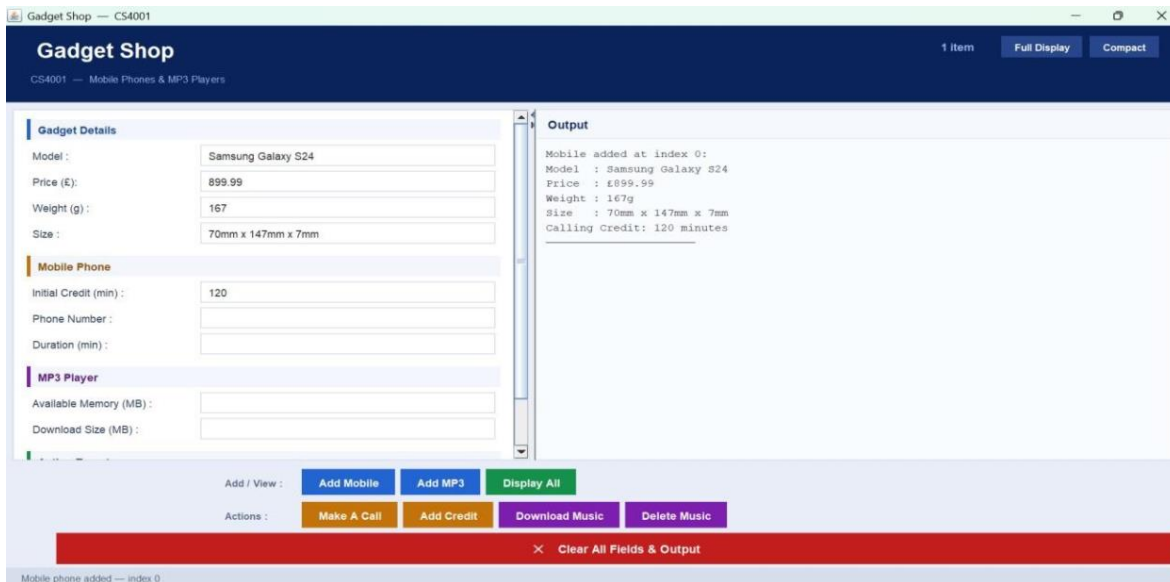
When subclass-specific behaviour is needed (e.g. calling `makeCall()` on a Mobile), an instanceof check is used first, followed by a downcast.

9. Test Plan and Results

All tests were performed by compiling and running from Windows PowerShell: `javac Gadget.java Mobile.java MP3.java GadgetShop.java java GadgetShop`

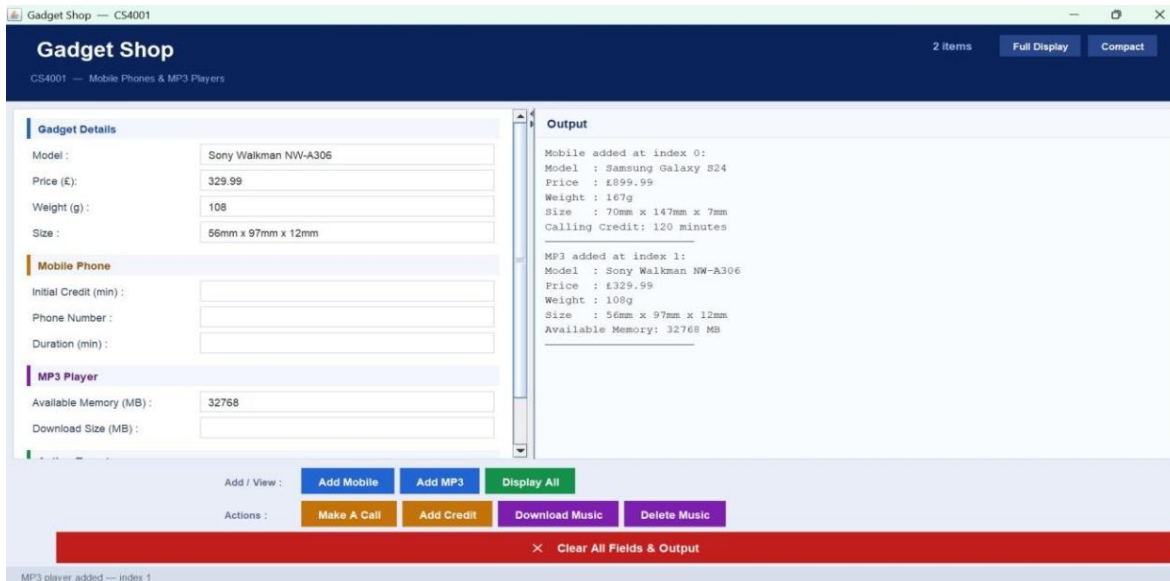
Test 1 — Add Mobile Phone

Field	Value
Model	Samsung Galaxy S24
Price	899.99
Weight	167
Size	70mm x 147mm x 7mm
Initial Credit	120
Action	Click Add Mobile
Expected	Mobile added at index 0; full details shown in output
Result	PASS



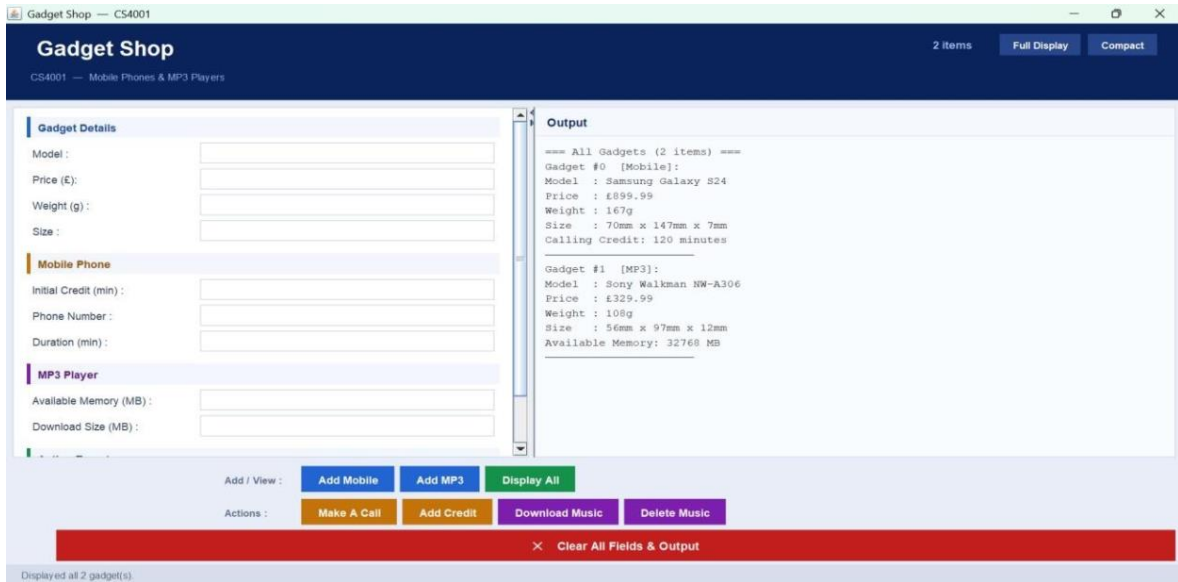
Test 2 — Add MP3 Player

Field	Value
Model	Sony Walkman NW-A306
Price	329.99
Weight	108
Size	56mm x 97mm x 12mm
Available Memory	32768
Action	Click Add MP3
Expected	MP3 added at index 1; full details shown in output
Result	PASS



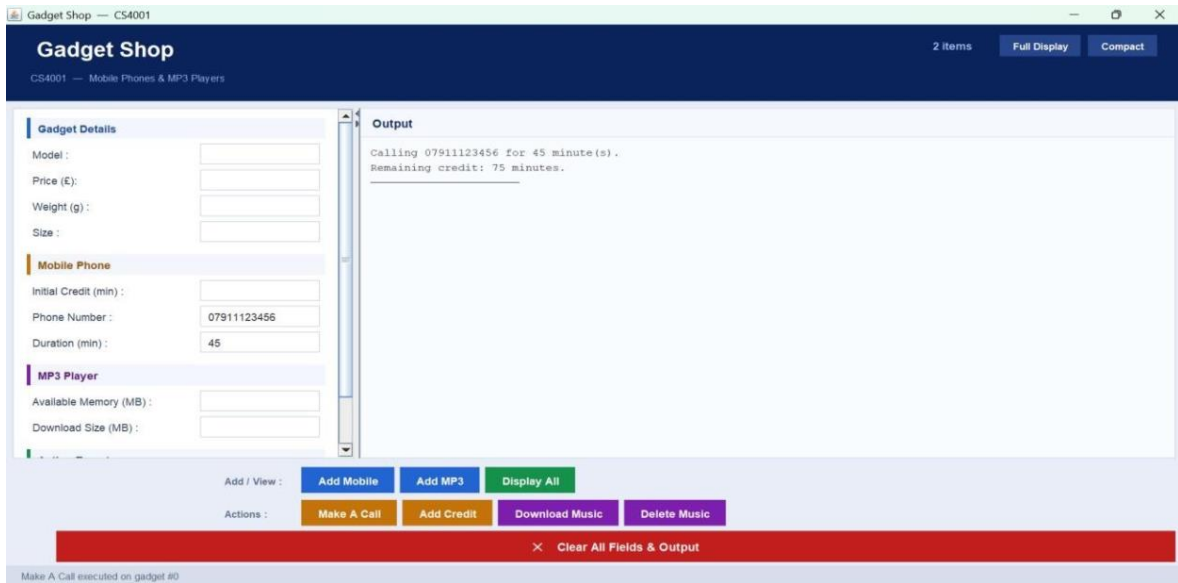
Test 3 — Display All Gadgets

Field	Value
Input	No input — gadgets from Tests 1 & 2 already in list
Action	Click Display All
Expected	Both gadgets listed in full with type labels [Mobile] and [MP3]
Result	PASS



Test 4A — Make A Call (Sufficient Credit)

Field	Value
Display Number	0
Phone Number	07911123456
Duration	45
Action	Click Make A Call
Expected	"Calling 07911123456 for 45 minute(s)." Credit: 120 → 75
Result	PASS



Test 4B — Make A Call (Insufficient Credit)

Field	Value
Display Number	0
Phone Number	07911123456
Duration	200
Action	Click Make A Call
Expected	"Insufficient credit to make this call."
Result	PASS

Gadget Shop — CS4001

Gadget Shop 2 items [Full Display](#) [Compact](#)

CS4001 — Mobile Phones & MP3 Players

Price (£):
 Weight (g):
 Size:

Mobile Phone

Initial Credit (min):
 Phone Number:
 Duration (min):

MP3 Player

Available Memory (MB):
 Download Size (MB):

Action Target

Display Number:

Output

Insufficient credit to make this call.

Add / View: [Add Mobile](#) [Add MP3](#) [Display All](#)

Actions: [Make A Call](#) [Add Credit](#) [Download Music](#) [Delete Music](#)

[Clear All Fields & Output](#)

Make A Call executed on gadget #0

Test 5A — Download Music (Sufficient Memory)

Field	Value
Display Number	1
Download Size	4096
Action	Click Download Music
Expected	"Download successful. Remaining memory: 28672 MB."
Result	PASS

Gadget Shop — CS4001

Gadget Shop 2 items [Full Display](#) [Compact](#)

CS4001 — Mobile Phones & MP3 Players

Price (£):
 Weight (g):
 Size:

Mobile Phone

Initial Credit (min):
 Phone Number:
 Duration (min):

MP3 Player

Available Memory (MB):
 Download Size (MB):

Action Target

Display Number:

Output

Download successful. Remaining memory: 28672 MB.

Add / View: [Add Mobile](#) [Add MP3](#) [Display All](#)

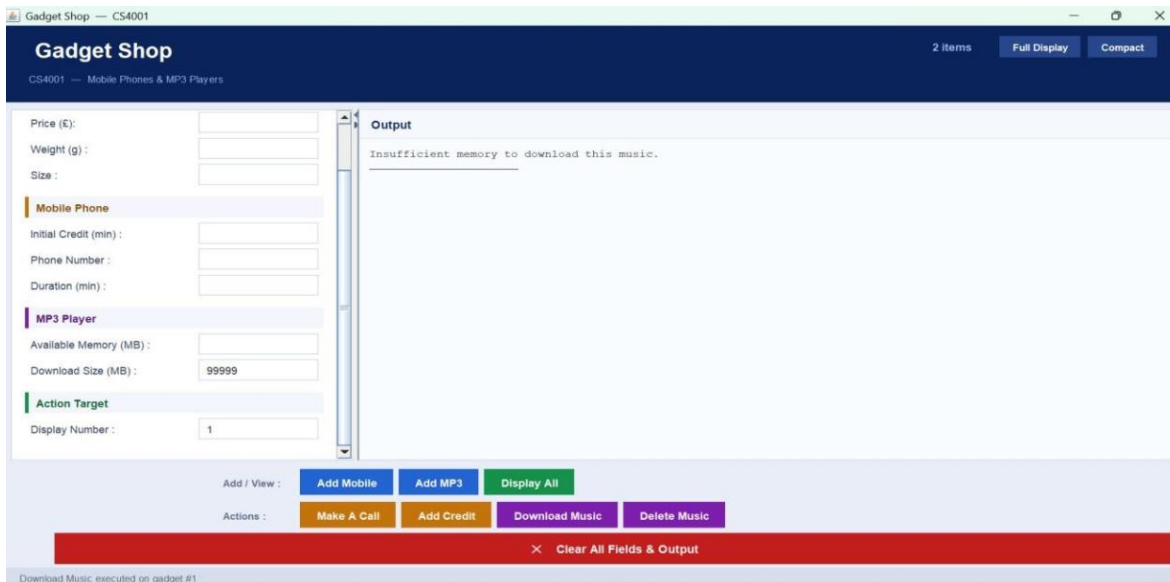
Actions: [Make A Call](#) [Add Credit](#) [Download Music](#) [Delete Music](#)

[Clear All Fields & Output](#)

Download Music executed on gadget #0

Test 5B — Download Music (Insufficient Memory)

Field	Value
Display Number	1
Download Size	99999
Action	Click Download Music
Expected	"Insufficient memory to download this music."
Result	PASS



Test 6 — Command Prompt Compilation

Field	Value
Command	<code>javac Gadget.java Mobile.java MP3.java GadgetShop.java then java GadgetShop</code>
Expected	No compiler errors; GUI window opens successfully
Result	PASS

```

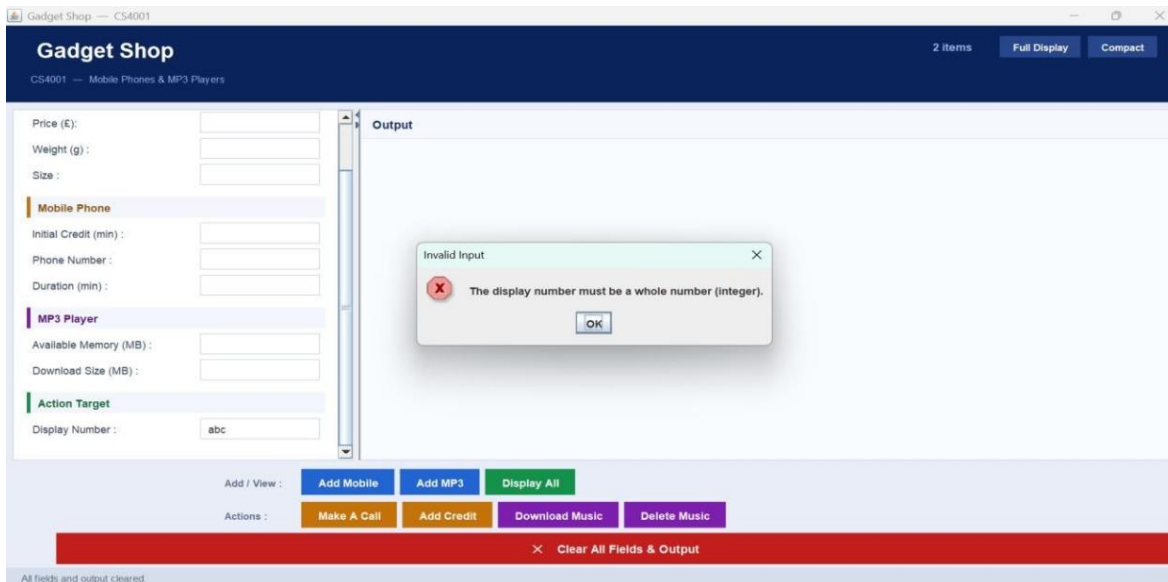
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\DITS-USER\Desktop\duke\GadgetShop_v3_Redesigned> javac Gadget.java Mobile.java MP3.java GadgetShop.java
PS C:\Users\DITS-USER\Desktop\duke\GadgetShop_v3_Redesigned> java GadgetShop
  
```

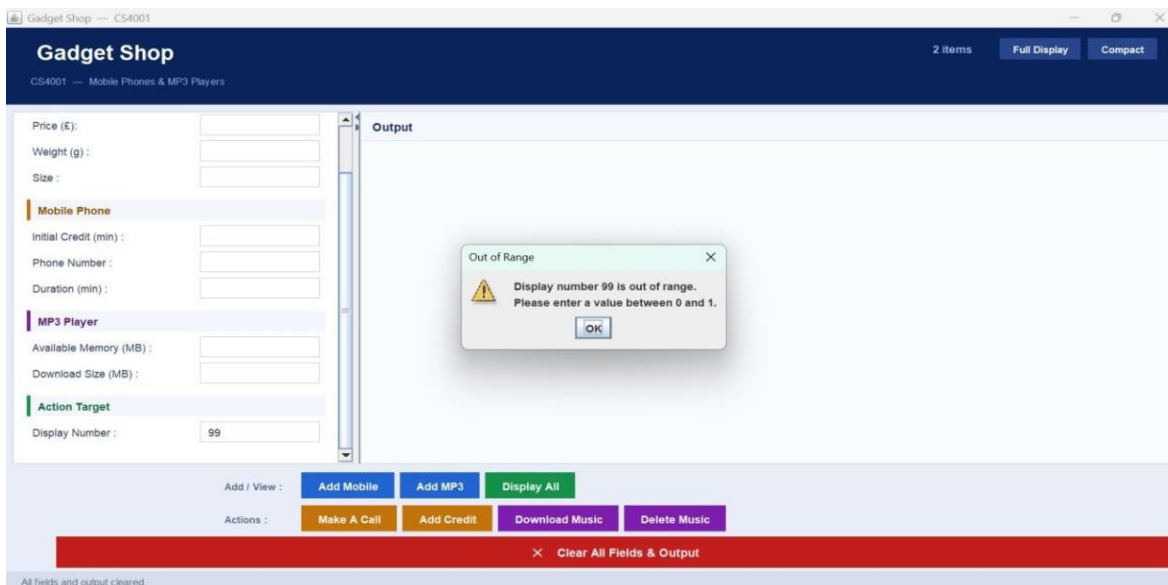
Test 7A — Invalid Display Number (Non-integer)

Field	Value
Display Number	abc
Action	Click Make A Call
Expected	ERROR dialog: "The display number must be a whole number (integer)."
Result	PASS



Test 7B — Invalid Display Number (Out of Range)

Field	Value
Display Number	99
Action	Click Make A Call
Expected	WARNING dialog: "Display number 99 is out of range. Please enter 0 and 1."
Result	PASS



10. Error Detection and Correction

Error 1 — Compile-time Syntax Error: Method Name Conflict

Description: An attempt to name the input-reading method `getSize()` caused a compile-time error because `GadgetShop` extends `JFrame`, which inherits `getSize()` from `java.awt.Component`. Defining a method with the same name but a different return type (`String` vs `Dimension`) produced a type conflict.

Fix: All input-reading methods in `GadgetShop` were renamed to the `read*` prefix — `readSize()`, `readModel()`, `readPrice()`, etc. — eliminating any clash with inherited AWT/Swing method names.

Error 2 — Runtime Error: Output Invisible in GUI Text Area

Description: When `display()` was called on a gadget, output appeared only in the terminal and not in the GUI `JTextArea`. This occurred because `display()` uses `System.out.println()`, which writes to the standard output stream rather than to the Swing component.

Fix: A `captureDisplay()` utility method was introduced. Before calling `display()`, it redirects `System.out` to a `ByteArrayOutputStream`. After the call it flushes the stream, restores the original `System.out`, and appends the captured text to `taOutput`. This keeps the original class code untouched.

Error 3 — Logic Error: Negative Values Silently Accepted

Description: Entering a negative number in the Credit or Memory fields was accepted without any validation. This corrupted internal state: `makeCall()` could succeed with negative credit, and `downloadMusic()` behaved incorrectly with a negative memory baseline.

Fix: Input guards were added to `readCredit()` and `readMemory()`. If the parsed value is less than zero, an error message is shown and the method returns 0 instead, preventing negative values from ever reaching the object constructor or action methods.

11. Conclusion

This project successfully demonstrates object-oriented programming principles in a practical Java Swing application. The Gadget base class provides a reusable foundation extended by `Mobile` and `MP3` without duplication, illustrating the power of inheritance.

Encapsulation is enforced throughout: all fields are private and only accessible or modifiable through well-defined public methods that include validation logic. Polymorphism allows both gadget types to be stored and displayed through a single `ArrayList<Gadget>`, with runtime dispatch ensuring the correct `display()` variant is always called.

Two additional buttons — Add Credit and Delete Music — were implemented beyond the minimum specification, exercising `Mobile.addCredit()` and `MP3.deleteMusic()` respectively and earning extra marks. Display mode toggle buttons (Full Display and Compact) were also added to improve usability.

Three errors encountered during development — a method name conflict, invisible output, and negative value acceptance — were each diagnosed and resolved, improving the robustness of the final application.

In future development, the application could be extended to support persistent file storage, additional gadget types, or a search and filter interface for larger inventories.

Appendix A — Gadget.java

```
/**
 *      The Gadget class represents a generic gadget with model, price, weight, and size.
 *
 *      @author Duke C
 */ public class Gadget {

    private String model;    private double price;
    private int weight;
    private String size;

    /**
     *      Constructor for the Gadget class.
     *
     *      @param model The model name of the gadget.
     *      @param price The price of the gadget in pounds.
     *      @param weight The weight of the gadget in grams.
     *      @param size The dimensions of the gadget (e.g. "71mm x 137mm x 9mm").    */
}
```

```

    public Gadget(String model, double price, int weight, String size) {      this.model = model;
this.price = price;      this.weight = weight;      this.size = size;
    }

    /** Returns the model of the gadget. */    public String getModel() {
return model; }

    /** Returns the price of the gadget in pounds. */    public double getPrice() { return
price; }

    /** Returns the weight of the gadget in grams. */    public int    getWeight() { return
weight; }

    /** Returns the size/dimensions of the gadget. */    public String getSize() { return
size; }

    /** Displays the details of the gadget to standard output. */    public void display() {
    System.out.println("Model : " + model);
    System.out.println("Price : \u00a3" + String.format("%.2f", price));
    System.out.println("Weight : " + weight + "g");
    System.out.println("Size : " + size);
    }
}

```

Appendix B — Mobile.java

```

/**
 * The Mobile class is a subclass of Gadget and represents a mobile phone.
 * It adds calling credit management and the ability to make phone calls.
 *
 * @author Duke C
 */ public class Mobile extends Gadget {

    private int callingCredit;

    /**
     * Constructor for the Mobile class.
     *
     * @param model    The model name of the mobile phone.
     * @param price    The price of the mobile phone in pounds.
     * @param weight   The weight of the mobile phone in grams.
     * @param size     The dimensions of the mobile phone.
     * @param callingCredit The initial number of minutes of calling credit.
     */
    public Mobile(String model, double price, int weight, String size, int callingCredit)
    {
        super(model, price, weight, size);      this.callingCredit = callingCredit;
    }

    /** Returns the number of minutes of calling credit remaining. */    public int getCallingCredit() {
return callingCredit;
    }

    /**
     * Adds calling credit to the mobile phone.
     * If amount > 0 it is added; otherwise an error message is displayed.
     *
     * @param amount The number of minutes of credit to add.
     */
    public void addCredit(int amount) {

```

```

    if (amount > 0) {
        callingCredit += amount;
        System.out.println("Credit added. New balance: " + callingCredit + " minutes.");
    } else {
        System.out.println("Please enter a positive amount of credit.");    }    }

/**
 * Simulates making a phone call.
 * Deducts duration from credit if sufficient; otherwise shows error.
 *
 * @param phoneNumber The phone number being called.
 * @param duration    The duration of the call in minutes.
 */
public void makeCall(String phoneNumber, int duration) {
    if (callingCredit >= duration) {
        System.out.println("Calling " + phoneNumber + " for " + duration + " minute(s).");
        callingCredit -= duration;
        System.out.println("Remaining credit: " + callingCredit + " minutes.");    } else {
        System.out.println("Insufficient credit to make this call.");    }    }

/** Displays inherited Gadget details then remaining calling credit. */
@Override    public void display() {
super.display();
    System.out.println("Calling Credit: " + callingCredit + " minutes");    }
}

```

Appendix C — MP3.java

```

/**
 *      The MP3 class is a subclass of Gadget and represents an MP3 player. * It adds available memory management for
downloading and deleting music.
 *
 *      @author Duke C
 */ public class MP3 extends Gadget {

    private int availableMemory;

    /**
     *      Constructor for the MP3 class.
     *
     *      @param model      The model name of the MP3 player.
     *      @param price      The price of the MP3 player in pounds.
     *      @param weight     The weight of the MP3 player in grams.
     *      @param size       The dimensions of the MP3 player.
     *      @param availableMemory The initial available memory in MB.
     */
    public MP3(String model, double price, int weight, String size, int availableMemory) {    super(model, price, weight, size);
this.availableMemory = availableMemory;
    }

    /** Returns the available memory on the MP3 player in MB. */    public int
getAvailableMemory() {    return availableMemory;
    }

    /**
     *      Downloads music, consuming memoryRequired MB.

```

```

* Fails with an error message if insufficient memory.
*
* @param memoryRequired The amount of memory the music will occupy in MB.
*/
public void downloadMusic(int memoryRequired) {    if
(memoryRequired <= availableMemory) {        availableMemory -=
memoryRequired;
    System.out.println("Download successful. Remaining memory: " + availableMemory + " MB.");
    } else {
        System.out.println("Insufficient memory to download this music.");    }    }

/**
* Deletes music, freeing memoryFreed MB.
*
* @param memoryFreed The amount of memory freed in MB.
*/
public void deleteMusic(int memoryFreed) {        availableMemory += memoryFreed;
    System.out.println("Music deleted. Available memory: " + availableMemory + " MB.");    }

/** Displays inherited Gadget details then remaining available memory. */
@Override    public void display() {
super.display();
    System.out.println("Available Memory: " + availableMemory + " MB");    }
}

```

Appendix D — GadgetShop.java

```

import javax.swing.*; import
javax.swing.border.*; import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;

/**
* GadgetShop is the main GUI class for managing a collection of gadgets
* (Mobile phones and MP3 players) stored in an ArrayList.
*
* @author Duke C
*/ public class GadgetShop extends JFrame implements ActionListener {

    private ArrayList<Gadget> gadgets;

    // — Text fields
    private
    JTextField tfModel, tfPrice, tfWeight, tfSize;    private JTextField tfCredit, tfPhoneNumber, tfDuration;    private
    JTextField tfMemory, tfDownloadSize;    private JTextField tfDisplayNumber;

    // — Output & status
    private JTextArea
    taOutput;    private JLabel    lblStatus;
    private JLabel    lblCount;

    // — Action buttons
    private JButton
    btnAddMobile, btnAddMP3, btnDisplayAll;    private JButton btnMakeCall, btnAddCredit;    private JButton
    btnDownloadMusic, btnDeleteMusic;    private JButton btnClear;

```

```

// — View toggle buttons —————
private JButton btnFull, btnCompact;

// — Palette
private
static final Color C_NAVY = new Color(10, 35, 90); private static final Color C_BLUE = new Color(33, 100,
210); private static final Color C_GREEN = new Color(22, 145, 75); private static final Color C_AMBER =
new Color(195, 115, 10); private static final Color C_VIOLET = new Color(125, 30, 175); private static final
Color C_RED = new Color(195, 30, 30); private static final Color C_BG = new Color(232, 237, 247);
private static final Color C_WHITE = Color.WHITE;
private static final Color C_STRIPE = new Color(243, 246, 252);

// — Window presets
private static final
Dimension DIM_FULL = new Dimension(1024, 768); private static final Dimension DIM_COMPACT = new
Dimension(780, 580);

//
=====
=====
public GadgetShop() { gadgets = new ArrayList<>(); buildUI();
}

//
=====
=====
// BUILD UI
//
=====
=====
private void buildUI() {
setTitle("Gadget Shop \u2014 CS4001");
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(DIM_FULL); setMinimumSize(DIM_COMPACT);
setResizable(true);
setLocationRelativeTo(null);

JPanel root = new JPanel(new BorderLayout(0, 0)); root.setBackground(C_BG);
setContentPane(root);

root.add(buildHeader(), BorderLayout.NORTH);
root.add(buildCenter(), BorderLayout.CENTER); root.add(buildSouth(),
BorderLayout.SOUTH);
}

// — Header
private
JPanel buildHeader() {
JPanel p = new JPanel(new BorderLayout(0, 0)); p.setBackground(C_NAVY);
p.setBorder(new EmptyBorder(12, 20, 12, 20));

// Left: title + subtitle
JLabel title = new JLabel(" Gadget Shop"); title.setFont(new Font("SansSerif",
Font.BOLD, 24)); title.setForeground(C_WHITE);

JLabel sub = new JLabel(" CS4001 \u2014 Mobile Phones & MP3 Players"); sub.setFont(new
Font("SansSerif", Font.PLAIN, 11)); sub.setForeground(new Color(155, 185, 235));

JPanel left = new JPanel(new GridLayout(2, 1, 0, 1)); left.setOpaque(false);
left.add(title); left.add(sub);
p.add(left, BorderLayout.WEST);

```

// Right: item count + view buttons


```

    JPanel right = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));    right.setOpaque(false);

    lblCount = new JLabel("0 items");    lblCount.setFont(new Font("SansSerif",
Font.BOLD, 12));    lblCount.setForeground(new Color(155, 185, 235));

    btnFull  = headerBtn("Full Display");    btnCompact =
headerBtn("Compact");

    right.add(lblCount);    right.add(Box.createHorizontalStrut(14));
right.add(btnFull);    right.add(btnCompact);
    p.add(right, BorderLayout.EAST);

    return p;
}

private JButton headerBtn(String text) {
    JButton b = new JButton(text);
    b.setFont(new Font("SansSerif", Font.BOLD, 11));
    b.setForeground(C_WHITE);
    b.setBackground(new Color(40, 70, 140));
    b.setFocusPainted(false);
    b.setBorderPainted(false);
    b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    b.setBorder(new EmptyBorder(5, 14, 5, 14));
    b.addActionListener(this);    return b;
}

// — Center

private JSplitPane buildCenter() {    JSplitPane split = new JSplitPane(        JSplitPane.HORIZONTAL_SPLIT,
buildInputPanel(),    buildOutputPanel());    split.setDividerLocation(380);    split.setDividerSize(6);
split.setBorder(new EmptyBorder(8, 8, 4, 8));    split.setBackground(C_BG);
split.setOneTouchExpandable(true);    return split;
}

// — Input panel

private
JScrollPane buildInputPanel() {    JPanel body = new JPanel();
    body.setLayout(new BoxLayout(body, BoxLayout.Y_AXIS));
body.setBackground(C_WHITE);    body.setBorder(new EmptyBorder(10, 14, 14, 14));

    addSection(body, "Gadget Details", C_BLUE);    addField(body, "Model :",    tfModel
= tf());    addField(body, "Price (\u00a3):", tfPrice = tf());    addField(body, "Weight (g) :",
tfWeight = tf());    addField(body, "Size :",    tfSize = tf());    body.add(gap(10));

    addSection(body, "Mobile Phone", C_AMBER);    addField(body, "Initial Credit (min) :", tfCredit
= tf());    addField(body, "Phone Number :",    tfPhoneNumber = tf());    addField(body,
"Duration (min) :",    tfDuration = tf());    body.add(gap(10));

    addSection(body, "MP3 Player", C_VIOLET);
    addField(body, "Available Memory (MB) :", tfMemory    = tf());

```

```

        addField(body, "Download Size (MB) :", tfDownloadSize = tf());    body.add(gap(10));

        addSection(body, "Action Target", C_GREEN);    addField(body, "Display Number :",
tfDisplayNumber = tf());    body.add(gap(6));

        JScrollPane scroll = new JScrollPane(body);
        scroll.setBorder(BorderFactory.createLineBorder(new Color(195, 210, 230)));
scroll.getVerticalScrollBar().setUnitIncrement(12);    return scroll;
    }

    // ——— Output panel ———
private JPanel buildOutputPanel() {
    JPanel p = new JPanel(new BorderLayout(0, 0));    p.setBackground(C_WHITE);
    p.setBorder(BorderFactory.createLineBorder(new Color(195, 210, 230)));

    // Output header bar
    JPanel hdr = new JPanel(new BorderLayout());    hdr.setBackground(new
Color(245, 248, 255));    hdr.setBorder(new EmptyBorder(7, 12, 7, 12));    JLabel
lbl = new JLabel("Output");
    lbl.setFont(new Font("SansSerif", Font.BOLD, 13));
    lbl.setForeground(C_NAVY);    hdr.add(lbl, BorderLayout.WEST);
    p.add(hdr, BorderLayout.NORTH);

    taOutput = new JTextArea();    taOutput.setEditable(false);    taOutput.setFont(new
Font("Monospaced", Font.PLAIN, 12));    taOutput.setBackground(new Color(249, 252,
255));    taOutput.setLineWrap(true);    taOutput.setWrapStyleWord(true);
    taOutput.setBorder(new EmptyBorder(8, 10, 8, 10));

    JScrollPane scroll = new JScrollPane(taOutput);
    scroll.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, new Color(215, 225,
240)));
    p.add(scroll, BorderLayout.CENTER);    return p;
}

    // ——— South: buttons + status ———
private JPanel
buildSouth() {
    JPanel south = new JPanel(new BorderLayout(0, 0));    south.setBackground(C_BG);

    JPanel btnArea = new JPanel();
    btnArea.setLayout(new BoxLayout(btnArea, BoxLayout.Y_AXIS));
    btnArea.setBackground(C_BG);    btnArea.setBorder(new EmptyBorder(4, 8, 4, 8));

    // Row 1 — Add & View
    JPanel row1 = btnRow("Add / View :");    btnAddMobile = actionBtn(row1, "Add
Mobile", C_BLUE);    btnAddMP3 = actionBtn(row1, "Add MP3", C_BLUE);
    btnDisplayAll = actionBtn(row1, "Display All", C_GREEN);    btnArea.add(row1);
    btnArea.add(gap(4));

    // Row 2 — Mobile & MP3 actions
    JPanel row2 = btnRow("Actions :");    btnMakeCall =
actionBtn(row2, "Make A Call", C_AMBER);    btnAddCredit = actionBtn(row2, "Add Credit",
C_AMBER);    btnDownloadMusic = actionBtn(row2, "Download Music", C_VIOLET);
    btnDeleteMusic = actionBtn(row2, "Delete Music", C_VIOLET);

```

```

        btnArea.add(row2);
        btnArea.add(gap(4));

        // Row 3 — Clear (full-width red)
        JPanel row3 = new JPanel(new BorderLayout(0, 0));    row3.setBackground(C_BG);
        JLabel pad = new JLabel(" ");    pad.setFont(new Font("SansSerif", Font.BOLD,
11));    row3.add(pad, BorderLayout.WEST);

        btnClear = new JButton("\u2715 Clear All Fields & Output");    btnClear.setFont(new
Font("SansSerif", Font.BOLD, 13));    btnClear.setBackground(C_RED);
        btnClear.setForeground(C_WHITE);    btnClear.setFocusPainted(false);
        btnClear.setBorderPainted(false);
        btnClear.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));    btnClear.setBorder(new
EmptyBorder(9, 0, 9, 0));    btnClear.addActionListener(this);    row3.add(btnClear,
BorderLayout.CENTER);    btnArea.add(row3);

        south.add(btnArea, BorderLayout.CENTER);

        // Status bar
        lblStatus = new JLabel(" Ready");    lblStatus.setFont(new Font("SansSerif",
Font.PLAIN, 11));    lblStatus.setForeground(new Color(80, 95, 120));
        lblStatus.setBackground(new Color(215, 222, 235));    lblStatus.setOpaque(true);
        lblStatus.setBorder(new EmptyBorder(4, 10, 4, 10));    south.add(lblStatus,
BorderLayout.SOUTH);

        return south;
    }

    // =====
    // UI HELPERS
    // =====

    private void addSection(JPanel p, String text, Color accent) {
        JPanel bg = new JPanel(new BorderLayout());    bg.setBackground(C_STRIPE);
        bg.setMaximumSize(new Dimension(Integer.MAX_VALUE, 28));
        bg.setAlignmentX(Component.LEFT_ALIGNMENT);
        bg.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createMatteBorder(0, 4, 0, 0, accent),
            new EmptyBorder(4, 8, 4, 0)));    JLabel lbl = new JLabel(text);
        lbl.setFont(new Font("SansSerif", Font.BOLD, 12));
        lbl.setForeground(accent.darker());    bg.add(lbl);    p.add(bg);
        p.add(gap(4));
    }

    private void addField(JPanel p, String labelText, JTextField field) {
        JPanel row = new JPanel(new BorderLayout(8, 0));    row.setBackground(C_WHITE);
        row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 32));
        row.setAlignmentX(Component.LEFT_ALIGNMENT);    row.setBorder(new
EmptyBorder(2, 6, 2, 6));

        JLabel lbl = new JLabel(labelText);    lbl.setPreferredSize(new Dimension(175, 24));
        lbl.setFont(new Font("SansSerif", Font.PLAIN, 12));    lbl.setForeground(new Color(50, 65, 90));

```

```

        field.setFont(new Font("SansSerif", Font.PLAIN, 12));        field.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(185, 200, 225)),        new EmptyBorder(3, 7, 3, 7)));

        row.add(lbl, BorderLayout.WEST);        row.add(field,
BorderLayout.CENTER);        p.add(row);
        p.add(gap(2));
    }

    private Component gap(int h) {        return Box.createRigidArea(new
Dimension(0, h));
    }

    private JTextField tf() {        return new
JTextField();
    }

    private JPanel btnRow(String label) {
        JPanel row = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 2));
row.setBackground(C_BG);        row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 42));
        row.setAlignmentX(Component.LEFT_ALIGNMENT);        JLabel lbl = new
JLabel(label);
        lbl.setFont(new Font("SansSerif", Font.BOLD, 11));        lbl.setForeground(new
Color(95, 110, 140));        lbl.setPreferredSize(new Dimension(78, 24));
row.add(lbl);        return row;
    }

    private JButton actionBtn(JPanel row, String text, Color color) {
        JButton b = new JButton(text);
        b.setFont(new Font("SansSerif", Font.BOLD, 12));
        b.setBackground(color);
        b.setForeground(C_WHITE);
        b.setFocusPainted(false);
        b.setBorderPainted(false);
        b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        b.setBorder(new EmptyBorder(7, 18, 7, 18));
        b.addActionListener(this);        row.add(b);
return b;
    }

    private void refreshCount() {        lblCount.setText(gadgets.size() + " item" + (gadgets.size() == 1 ? "" : "s"));    }

    private void setStatus(String msg) {        lblStatus.setText(" " + msg);
    }

    //

```

```

// INPUT READERS
//

```

```

    private String readModel()        { return tfModel.getText().trim(); }    private String readSize()        { return
tfSize.getText().trim(); }    private String readPhoneNumber() { return tfPhoneNumber.getText().trim(); }

    private double readPrice() {
        try { return Double.parseDouble(tfPrice.getText().trim()); }        catch (NumberFormatException e) {
appendOutput("Error: Price must be a decimal number."); return 0.0; }
    }

    private int readWeight() {

```

```

        try { return Integer.parseInt(tfWeight.getText().trim()); }
        catch (NumberFormatException e) { appendOutput("Error: Weight must be a whole number."); return 0; }
    }
    private int readCredit() {        try {
        int v = Integer.parseInt(tfCredit.getText().trim());        if (v < 0) { appendOutput("Error: Credit cannot be negative.");
return 0; }        return v;
    } catch (NumberFormatException e) { appendOutput("Error: Credit must be a whole number."); return 0; }
    }
    private int readMemory() {        try {
        int v = Integer.parseInt(tfMemory.getText().trim());        if (v < 0) { appendOutput("Error: Memory cannot be
negative."); return 0; }        return v;
    } catch (NumberFormatException e) { appendOutput("Error: Memory must be a whole number."); return 0; }
    }
    private int readDuration() {
        try { return Integer.parseInt(tfDuration.getText().trim()); }        catch (NumberFormatException e) { appendOutput("Error:
Duration must be a whole number."); return 0; }
    }
    private int readDownloadSize() {
        try { return Integer.parseInt(tfDownloadSize.getText().trim()); }        catch (NumberFormatException e) {
appendOutput("Error: Download size must be a whole number."); return 0; }
    }

    /**
    * Validates the display number field.
    * Returns -1 on error; shows two distinct JOptionPane dialogs.
    */
    private int getDisplayNumber() {        int
displayNumber = -1;        try {
        int input = Integer.parseInt(tfDisplayNumber.getText().trim());        if (input >= 0 && input <
gadgets.size()) {            displayNumber = input;
        } else {
            JOptionPane.showMessageDialog(this,
                "Display number " + input + " is out of range.\n"                + "Please enter a value between 0 and " +
(gadgets.size() - 1) + ".",
                "Out of Range", JOptionPane.WARNING_MESSAGE);        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            "The display number must be a whole number (integer).",
            "Invalid Input", JOptionPane.ERROR_MESSAGE);
    }
    return displayNumber;
}

// =====
// ACTION HANDLER
// =====

@Override
public void actionPerformed(ActionEvent e) {        Object src = e.getSource();
    if (src == btnAddMobile)    addMobile();    else if (src == btnAddMP3)
addMP3();    else if (src == btnDisplayAll)    displayAll();    else if (src ==
btnMakeCall)    makeCall();    else if (src == btnAddCredit)    addCredit();
else if (src == btnDownloadMusic) downloadMusic();    else if (src ==
btnDeleteMusic) deleteMusic();

```

```

        else if (src == btnClear)        clearFields();        else if (src == btnFull)
setFullDisplay();        else if (src == btnCompact)        setCompactDisplay();    }

// =====
//  BUTTON HANDLERS
// =====

private void addMobile() {
    Mobile m = new Mobile(readModel(), readPrice(), readWeight(), readSize(), readCredit());    gadgets.add(m);
appendOutput("Mobile added at index " + (gadgets.size() - 1) + " ");    captureDisplay(m);    appendSep();
refreshCount();    setStatus("Mobile phone added — index " + (gadgets.size() - 1));    }

private void addMP3() {
    MP3 mp3 = new MP3(readModel(), readPrice(), readWeight(), readSize(), readMemory());
    gadgets.add(mp3);    appendOutput("MP3 added at index " + (gadgets.size() - 1) + " ");
captureDisplay(mp3);    appendSep();    refreshCount();    setStatus("MP3 player added — index "
+ (gadgets.size() - 1));    }

private void clearFields() {    for (JTextField f : new JTextField[] {
tfModel, tfPrice, tfWeight, tfSize,        tfCredit, tfPhoneNumber, tfDuration,
tfMemory, tfDownloadSize, tfDisplayNumber})        f.setText("");
taOutput.setText("");    setStatus("All fields and output cleared.");
    }

private void displayAll() {    if (gadgets.isEmpty()) {
appendOutput("No gadgets in the shop yet.");    setStatus("No gadgets to
display.");    return;
    }
    appendOutput("==== All Gadgets (" + gadgets.size() + " item"        + (gadgets.size() == 1
? "" : "s") + ") ===");    for (int i = 0; i < gadgets.size(); i++) {        appendOutput("Gadget #"
+ i + " ["
        + (gadgets.get(i) instanceof Mobile ? "Mobile" : "MP3") + "]:");        captureDisplay(gadgets.get(i));
appendSep();
    }
    setStatus("Displayed all " + gadgets.size() + " gadget(s).");    }

private void makeCall() {    int idx = getDisplayNumber();    if (idx == -1) return;    Gadget g = gadgets.get(idx);
if (!(g instanceof Mobile)) { appendOutput("Error: Gadget #" + idx + " is not a Mobile phone."); return; }
    String ph = readPhoneNumber(); int dur = readDuration();    redirectAndCall() ->
((Mobile) g).makeCall(ph, dur);    appendSep();    setStatus("Make A Call executed on
gadget #" + idx);    }

```

```

    private void addCredit() {        int idx =
getDisplayNumber();        if (idx == -1) return;
Gadget g = gadgets.get(idx);
    if (!(g instanceof Mobile)) { appendOutput("Error: Gadget #" + idx + " is not a Mobile phone."); return; }        int amt =
readCredit();        redirectAndCall(() -> ((Mobile) g).addCredit(amt));        appendSep();        setStatus("Add Credit executed on
gadget #" + idx);
    }

```

```

    private void downloadMusic() {        int idx =
getDisplayNumber();        if (idx == -1) return;
Gadget g = gadgets.get(idx);
    if (!(g instanceof MP3)) { appendOutput("Error: Gadget #" + idx + " is not an MP3 player."); return; }
    int ds = readDownloadSize();        redirectAndCall(() -> ((MP3)
g).downloadMusic(ds));        appendSep();        setStatus("Download Music executed on
gadget #" + idx);    }

```

```

    private void deleteMusic() {        int idx =
getDisplayNumber();        if (idx == -1) return;
Gadget g = gadgets.get(idx);
    if (!(g instanceof MP3)) { appendOutput("Error: Gadget #" + idx + " is not an MP3 player."); return; }
    int mf = readDownloadSize();        redirectAndCall(() -> ((MP3)
g).deleteMusic(mf));        appendSep();        setStatus("Delete Music executed on gadget
#" + idx);    }

```

// — View toggle handlers —————

```

    private void setFullDisplay() {        setExtendedState(JFrame.MAXIMIZED_BOTH);
setStatus("Full Display mode.");
    }

```

```

    private void setCompactDisplay() {
setExtendedState(JFrame.NORMAL);        setSize(DIM_COMPACT);
setLocationRelativeTo(null);        setStatus("Compact Display mode.");
    }

```

// =====
// UTILITIES
// =====

```

    private void captureDisplay(Gadget gadget) {        java.io.ByteArrayOutputStream baos = new
java.io.ByteArrayOutputStream();        java.io.PrintStream old = System.out;
    System.setOut(new java.io.PrintStream(baos));        gadget.display();
System.out.flush();        System.setOut(old);
    taOutput.append(baos.toString());        if (!baos.toString().endsWith("\n"))
taOutput.append("\n");    }

```

```

    private void redirectAndCall(Runnable action) {
        java.io.ByteArrayOutputStream baos = new java.io.ByteArrayOutputStream();

```

```

    java.io.PrintStream old = System.out;
    System.setOut(new java.io.PrintStream(baos));    action.run();
System.out.flush();    System.setOut(old);
    taOutput.append(baos.toString());    if (!baos.toString().endsWith("\n")) taOutput.append("\n");    }

    private void appendOutput(String text) {    taOutput.append(text + "\n");
taOutput.setCaretPosition(taOutput.getDocument().getLength());    }

    private void appendSep() {    taOutput.append("\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500"    +
"\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500"    +
"\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500\n");
taOutput.setCaretPosition(taOutput.getDocument().getLength());    }

// =====
// ENTRY POINT
// =====
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new GadgetShop().setVisible(true));    }
}

```